Universidad ORT Uruguay
Facultad de Ingeniería
Escuela de Tecnología

OBLIGATORIO

Desarrollo Web Asistido por IA



[Camilo Pereira Garcia – 373693]



[Ignacio Federico Alvez Perera – 361351]

[M3A]

Docente: [Lucas López]

[ATI]

Índice

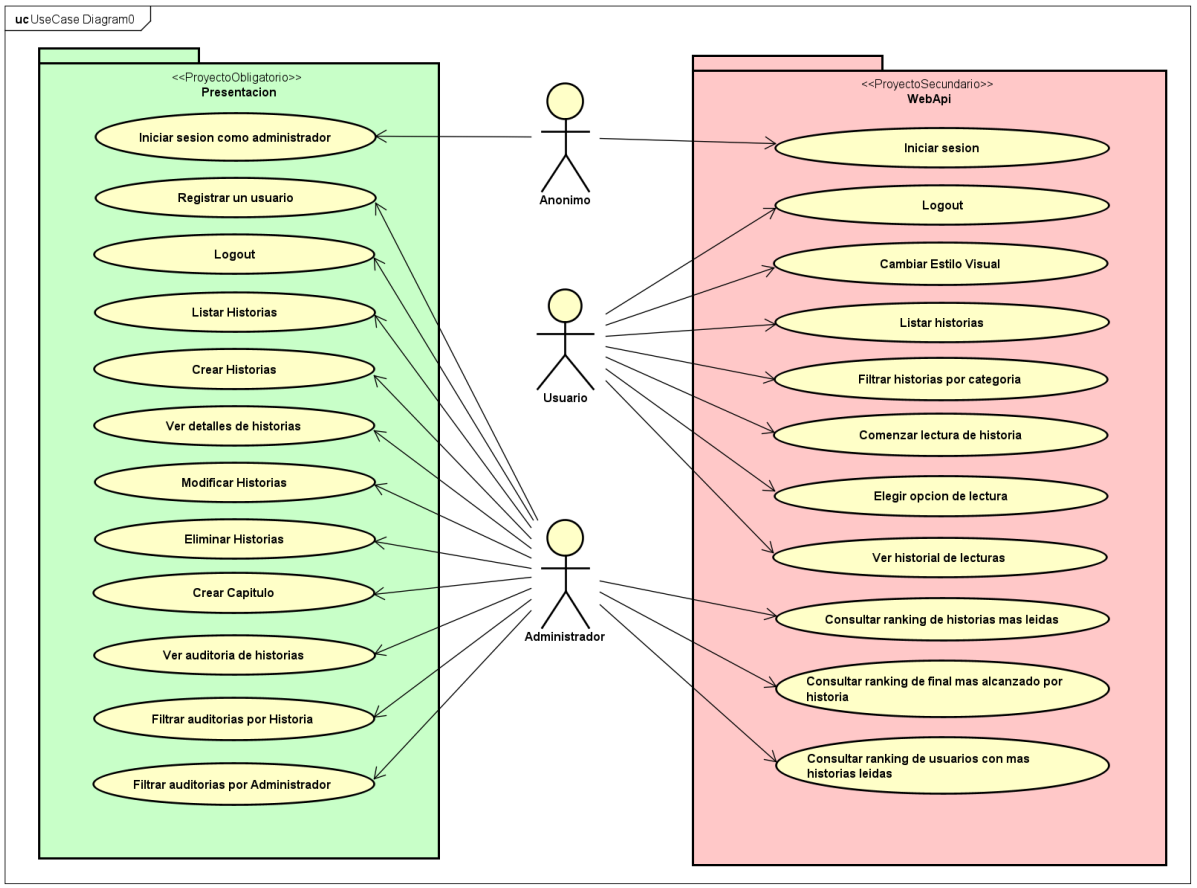
Links del Obligatorio **3**
Diagrama de casos de uso **4**
Descripción narrativa de los casos de uso **4**
RF07 - Lectura de historia 4
RF08 - Historial de lecturas 5
Diseño de casos de prueba – RF09: Ranking de historias más leídas **6**
Diagrama de clases (UML) **9**
Generación de datos de prueba (SQL) **10**
Prompt 10
Respuesta 10
Contenido e implementación 10
Notas técnicas 10
Aclaración 11
Investigación realizada para implementar la selección de estilos **11**
Prompt 11
Respuesta 11
1. Objetivo 11
2. Flujo funcional 11
2.1 Carga inicial 11
2.2 Cambio de estilo 12
2.3 Persistencia durante la navegación 12
2.4 Comportamiento durante logout 12
3. Implementación técnica 13
3.1 Configuración de sesión 13
3.2 Variables y validación 13
3.3 Acción CambiarEstilo 14
3.4 Aplicación en el layout 14
3.5 Estilos CSS 14
4. Archivos de la implementación 15
5. Seguridad 15
6. Pruebas manuales 15
Prueba 1: Estilo predeterminado 15
Prueba 2: Activación del estilo oscuro 15
Prueba 3: Persistencia después del logout 16
Prueba 4: Regreso al estilo claro 16
Aclaración 16
Evidencia de pruebas de funcionamiento de la API con Swagger **16**
Autenticación 16
Login 16
Resumen de resultados 17

Evidencia detallada por endpoint 18
Listar categorías — GET /api/Categoría 18
Listar historias — GET /api/Historia 18
Consultar catálogo por categoría — GET /api/Historia/Catálogo 19
Ranking de historias — GET /api/Historia/Ranking 20
Consultar finales de una historia — GET /api/Historia/{id}/Finales 21
Consultar una historia por ID — GET /api/Historia/{id} 21
Listar lecturas — GET /api/Lectura 22
Iniciar una lectura — POST /api/Lectura 23
Consultar una lectura por ID — GET /api/Lectura/{id} 24
Ranking de usuarios — GET /api/Lectura/RankingUsuarios 25
Historial de lecturas por usuario y período — GET /api/Lectura/Historial 25
Consultar un capítulo de la lectura — GET /api/Lectura/{id}/Capitulo/{idCapitulo} 26
Avanzar en la lectura — POST /api/Lectura/{id}/Avanzar 26
Listar usuarios — GET /api/Usuario 28
Consultar un usuario por ID — GET /api/Usuario/{id} 29

Links del Obligatorio

Repo Github	https://github.com/Camilo4per/ProyectoObligatorio2026
App de Lectura de Historias	http://lecturawebm3c.somee.com/
Endpoint WebApi	http://apiwebm3c.somee.com/api/

Diagrama de casos de uso



Descripción narrativa de los casos de uso

RF07 - Lectura de historia

El usuario autenticado accede a la opción “Leer historia”. El sistema muestra un listado de las historias publicadas y permite filtrarlas por categoría.

El usuario selecciona una historia e inicia una nueva lectura. El sistema comienza el recorrido desde el capítulo inicial. En cada capítulo intermedio, el usuario visualiza el contenido y selecciona una de las opciones disponibles para avanzar al capítulo correspondiente.

El recorrido continúa hasta que el usuario alcanza uno de los posibles finales de la historia. En ese momento, el sistema muestra el desenlace alcanzado y lo registra asociado a la lectura del usuario.

Una vez alcanzado un final, la lectura se considera terminada. Si el usuario desea explorar un camino diferente de la misma historia, deberá iniciar una nueva lectura desde el capítulo inicial.

Precondiciones

- El usuario debe estar autenticado.
- Debe existir al menos una historia publicada.
- La historia seleccionada debe tener un capítulo inicial.
- Los capítulos intermedios deben contar con opciones de navegación válidas.

Resultado esperado

El usuario recorre la historia de forma interactiva desde el capítulo inicial hasta un final, y el sistema registra el desenlace alcanzado. Cada nuevo intento de recorrer un camino diferente genera una nueva lectura.

RF08 - Historial de lecturas

El usuario autenticado accede a la opción “Historial de lecturas” e indica el mes y el año que desea consultar. El sistema busca las lecturas realizadas por el usuario durante el período seleccionado y muestra un listado con la siguiente información:

- Historia leída.
- Estado de la lectura: completada o no completada.
- Final alcanzado, en caso de que la lectura haya sido completada.

Si el usuario no realizó ninguna lectura durante el mes y año indicados, el sistema deberá mostrar un mensaje descriptivo informando que no existen lecturas para ese período.

Precondiciones

- El usuario debe estar autenticado.
- El mes y el año seleccionados deben ser válidos.

Resultado esperado

El sistema muestra únicamente las lecturas correspondientes al usuario logueado y al período seleccionado, indicando su estado y, cuando corresponda, el final alcanzado.

Diseño de casos de prueba – RF09: Ranking de historias más leídas

Listado de historias

ordenado por cantidad de lecturas completadas, de mayor a menor. Las historias

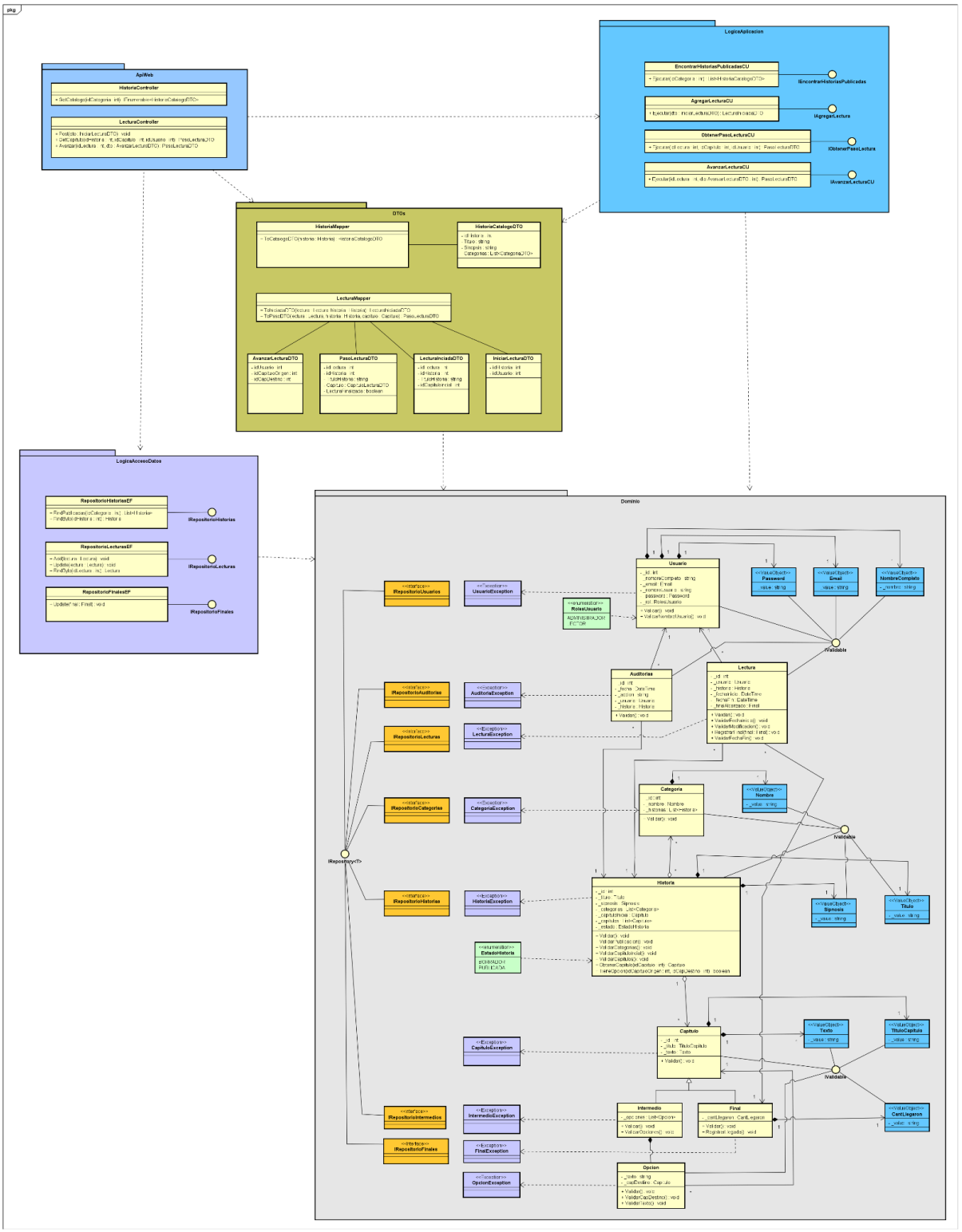
nunca leídas no deben aparecer en el listado.

ID	Escenario	Precondiciones	Pasos	Resultado esperado
RF08-CP01	Visualizar el ranking con historias publicadas	Existen historias con lecturas completadas	1. Iniciar sesión. 2. Acceder al Ranking	Se muestra el ranking con las historias disponibles.
RF08-CP02	Verificar el orden descendente	Historia A tiene 5 lecturas, B 3 y C 1	1. Acceder al ranking	Las historias aparecen ordenadas: A, B y C.
RF08-CP03	Verificar la posición de cada historia	El ranking contiene varias historias	1. Acceder al ranking	Las posiciones se muestran consecutivamente comenzando en 1.
RF08-CP04	Verificar la cantidad mostrada	Existen datos conocidos de	1. Acceder al ranking	La cantidad mostrada coincide con las lecturas

ID	Escenario	Precondiciones	Pasos	Resultado esperado
		lecturas completadas		completadas reales de cada historia.
RF08-CP05	Excluir lecturas no completadas	Historia A tiene lecturas completadas y no completadas	1. Acceder al ranking	Las lecturas no completadas no incrementa el contador del ranking.
RF08-CP06	No mostrar historias sin lecturas completadas	Historia D solo tiene lecturas no completadas	1. Acceder al ranking	Historia D no aparece en el listado.
RF08-CP07	Ranking sin resultados	No existen historias con lecturas completadas	1. Acceder al ranking	Se muestra el mensaje: "No existen historias con lecturas completadas."
RF08-CP08	Acceso sin autenticación	No hay una sesión iniciada	1. Intentar acceder directamente al ranking	El sistema redirige al usuario hacia la pantalla de inicio de sesión.
RF08-CP09	Acceso con rol no autorizado	El usuario está autenticado, pero no posee el rol habilitado	1. Acceder al ranking	El sistema impide el acceso y redirige a la pantalla correspondiente.
RF08-CP10	Error al consultar el servicio	El servicio de ranking no está disponible o devuelve un error	1. Acceder al ranking	Se muestra un mensaje indicando que no fue posible obtener el ranking.
RF08-CP11	Títulos de historias repetidos	Existen historias diferentes con el mismo título	1. Acceder al ranking	Cada historia se muestra como un registro independiente.
RF08-CP12	Empate entre historias	Dos o más historias tienen la misma cantidad de lecturas	1. Acceder al ranking	Las historias aparecen con la misma cantidad. Se muestran en el

ID	Escenario	Precondiciones	Pasos	Resultado esperado
				orden en que la API las devuelve.

Diagrama de clases (UML)



Generación de datos de prueba (SQL)

Prompt

Necesito que me generes un script sql con el nombre ScriptObligatorio que cumpla con el siguiente punto: E. Scripts SQL con los juegos de datos de prueba (mínimo 10 registros por tabla, esquemas inserts, incluyendo precargas y datos que surgen del uso del sistema). Quiero que las historias que generes tengan varias opciones y finales.

Respuesta

Contenido e implementación

- **Esquema:** Completo y totalmente compatible con SQL Server y EF Core.
- **Tablas incluidas:** Usuarios, Categorias, Historias, Capitulos, Opcion, CategoriaHistoria, Lecturas y Auditorias.
- **Carga de datos:**
    - Mínimo de 10 registros por tabla.
    - 10 historias interactivas.
    - 40 capítulos.
    - 40 opciones de navegación.
    - Varias rutas y dos finales por historia.
    - Lecturas en curso y finalizadas.
    - Datos de auditoría.
- **Verificación:** Consultas finales para validar cantidades, opciones y finales.

Notas técnicas

- El script utiliza IDENTITY_INSERT para mantener IDs deterministas y contempla la relación entre historias y capítulos iniciales.

- La ejecución contra SQL Server queda pendiente de una conexión/base de datos configurada.

Aclaración
Se realizaron cambios a todo lo conseguido por la ia para adecuarlo mejor a nuestro proyecto y conocimientos.

Investigación realizada para implementar la selección de estilos

Prompt

Quiero un plan para implementar dos estilos visuales, quiero que uses variables de session, que no se borren al logout, que haya un botón en el menú del layout para cambiar entre un estilo claro/oscuro y que documentes el paso a paso de su creación: RF12 – Selección de estilo de lectura.

Respuesta

1. Objetivo

RF12 permite que el usuario seleccione entre dos estilos visuales de lectura:

- Estilo claro.
- Estilo oscuro.

La preferencia se guarda en la sesión HTTP y se aplica desde el layout compartido de ProyectoSecundario.

La implementación utiliza el patrón actual del proyecto, basado en controladores MVC y vistas Razor. No se agregó una entidad de base de datos ni un sistema de persistencia adicional.

2. Flujo funcional

2.1 Carga inicial

1. El usuario solicita cualquier vista que utilice Views/Shared/_Layout.cshtml.
2. El layout consulta la variable de sesión EstiloLectura.
3. Si el valor es Oscuro, se asigna la clase CSS tema-oscuro al elemento body.

4. Para cualquier otro valor, incluido un valor inexistente, se asigna tema-claro.
5. El menú muestra la acción opuesta al estilo actual:
    - En estilo claro: Activar modo oscuro.
    - En estilo oscuro: Activar modo claro.

2.2 Cambio de estilo

1. El usuario presiona el botón del menú.
2. El formulario envía una petición POST a Auth/CambiarEstilo.
3. El formulario incluye el token antiforgery.
4. Se envían dos valores:
    - estilo: el estilo al que se desea cambiar.
    - returnUrl: la ruta actual para volver a la página donde se realizó la acción.
5. AuthController.CambiarEstilo valida el estilo recibido.
6. Si el valor es Oscuro, se guarda Oscuro; cualquier otro valor se normaliza a Claro.
7. La preferencia se guarda en la sesión mediante SetString.
8. Si returnUrl es una URL local válida, se redirige allí. En caso contrario, se redirige a la pantalla de login.
9. El layout se vuelve a renderizar con la nueva clase CSS.

2.3 Persistencia durante la navegación

La variable permanece disponible mientras la sesión HTTP esté vigente. Por eso el usuario puede navegar entre historias, rankings, lecturas y el historial sin perder el estilo seleccionado.

2.4 Comportamiento durante logout

El logout de AuthController elimina únicamente las variables relacionadas con la autenticación:

- token.
- Rol.
- NombreUsuario.
- IdUsuario.

No se ejecuta HttpContext.Session.Clear(), ya que esa operación también eliminaría EstiloLectura.

Como resultado:

1. El usuario deja de estar autenticado.
2. Las opciones protegidas desaparecen del menú.
3. El estilo visual seleccionado permanece en la sesión.
4. Al volver a iniciar sesión, el estilo continúa aplicado.

La preferencia se conserva durante el logout, pero sigue dependiendo del vencimiento normal de la sesión configurado en Program.cs.

3. Implementación técnica

3.1 Configuración de sesión

Program.cs registra una caché distribuida en memoria y la sesión:p
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
options.IdleTimeout = TimeSpan.FromMinutes(30);
options.Cookie.HttpOnly = true;
options.Cookie.IsEssential = true;
});

None

El middleware se ejecuta después de routing y antes de autorización:

```csharp
app.UseRouting();
app.UseSession();
app.UseAuthorization();
```

3.2 Variables y validación

AuthController define una clave y los valores válidos:

```
None
private const string ClaveEstiloLectura = "EstiloLectura";
private const string EstiloClaro = "Claro";
private const string EstiloOscuro = "Oscuro";
```

La acción de cambio solo acepta explícitamente Oscuro. Cualquier otro valor se convierte en Claro:

```
None
string estiloNormalizado = estilo == EstiloOscuro
    ? EstiloOscuro
    : EstiloClaro;
```

Esto evita que un valor recibido desde el cliente se convierta directamente en una clase CSS arbitraria.

3.3 Acción CambiarEstilo

La acción se implementa como POST porque modifica el estado de la sesión:

```
None
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult CambiarEstilo(string estilo, string? returnUrl)
```

Después de guardar la preferencia, Url.IsLocalUrl valida que el retorno no redirija a un sitio externo.

3.4 Aplicación en el layout

El layout interpreta la sesión y genera una clase controlada:r
string estiloLectura = Context.Session.GetString("EstiloLectura") == "Oscuro"
? "Oscuro"
: "Claro";

string claseEstilo = estiloLectura == "Oscuro"
? "tema-oscuro"
: "tema-claro";

```
None

La clase se aplica al `body`:

```razor
<body class="@claseEstilo">
```

El botón se encuentra dentro de un formulario Razor que apunta a Auth/CambiarEstilo y contiene el token antiforgery.

3.5 Estilos CSS

wwwroot/css/site.css define variables para cada tema:

- Color de fondo.
- Color de superficie del navbar.
- Color del texto.
- Color de bordes.
- Color de enlaces.
- Color de controles de formulario.

Las reglas se activan mediante:
```

```
None
body.tema-claro { }
body.tema-oscuro { }
```

También se adaptan el navbar, los enlaces, los formularios, las tablas, el footer y el botón responsive de navegación.

4. Archivos de la implementación

- ProyectoSecundario/ProyectoSecundario/Program.cs
    - Configuración y middleware de sesión existente.
- ProyectoSecundario/ProyectoSecundario/Controllers/AuthController.cs
    - Cambio de estilo y logout selectivo.
- ProyectoSecundario/ProyectoSecundario/Views/Shared/_Layout.cshtml
    - Lectura de la sesión, botón y aplicación del tema.
- ProyectoSecundario/ProyectoSecundario/wwwroot/css/site.css
    - Variables y reglas de los estilos claro y oscuro.
- ProyectoSecundario/docs/RF12-Seleccion-estilo-lectura.md
    - Documentación funcional y técnica del requisito.

5. Seguridad

La implementación contempla las siguientes medidas:

- El cambio de estilo utiliza POST.
- El formulario incluye @Html.AntiForgeryToken().
- El estilo recibido se normaliza contra una lista limitada de valores.
- La URL de retorno se valida con Url.IsLocalUrl.
- El logout elimina los datos de autenticación sin eliminar la preferencia visual.

6. Pruebas manuales

Prueba 1: Estilo predeterminado

1. Abrir la aplicación con una sesión nueva.
2. Comprobar que se visualiza el estilo claro.
3. Comprobar que el botón indica Activar modo oscuro.

Prueba 2: Activación del estilo oscuro

1. Presionar Activar modo oscuro.
2. Comprobar el cambio de fondo, navbar, texto y controles.
3. Navegar a otra sección.
4. Comprobar que el estilo oscuro se mantiene.

Prueba 3: Persistencia después del logout

1. Activar el estilo oscuro.
2. Presionar Logout.
3. Comprobar que desaparecen las opciones autenticadas.
4. Comprobar que el estilo oscuro sigue aplicado.
5. Volver a iniciar sesión.
6. Comprobar que el estilo oscuro se conserva.

Prueba 4: Regreso al estilo claro

1. Presionar Activar modo claro.
2. Comprobar que la interfaz vuelve al estilo claro.
3. Navegar a otra sección.
4. Comprobar que el estilo claro se mantiene.

Aclaración

Se realizaron cambios a todo lo conseguido por la ia para adecuarlo mejor a nuestro proyecto y conocimientos.

Evidencia de pruebas de funcionamiento de la API con Swagger

Autenticación

Login

Solicitud

POST http://apiwebm3c.somee.com/api/Usuario/Login
Content-Type: application/json

```
JSON
{
  "nombreUsuario": "admin01",
  "password": "HashDemo_Admin01"
}
```

Respuesta — 200 OK

```
JSON
{
```

```
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...<JWT truncado en el
documento>",
  "usuario": {
    "id": 1,
    "nombre": "Ana",
    "apellido": "Administrador",
    "nombreUsuario": "admin01",
    "rol": 0,
    "email": "ana.admin@obligatorio.test",
    "password": "HashDemo_Admin01"
  }
}
```


Resumen de resultados

#	Método	Endpoint	Código obtenido	Resultado
1	GET	/api/Categoria	200 OK	Correcto
2	GET	/api/Historia	200 OK	Correcto
3	GET	/api/Historia/Catalogo?idCategoria=1	200 OK	Correcto
4	GET	/api/Historia/Ranking	200 OK	Correcto
5	GET	/api/Historia/1/Finales	200 OK	Correcto
6	GET	/api/Historia/1	200 OK	Correcto
7	GET	/api/Lectura	200 OK	Correcto
8	POST	/api/Lectura	201 Created	Correcto
9	GET	/api/Lectura/27?idUsuario=10	200 OK	Correcto
10	GET	/api/Lectura/RankingUsuarios	200 OK	Correcto
11	GET	/api/Lectura/Historial?idUsuario=10&mes=11&anio=2026	200 OK	Correcto
12	GET	/api/Lectura/27/Capitulo/1?idUsuario=10	200 OK	Correcto
13	POST	/api/Lectura/27/Avanzar	200 OK	Correcto

#	Método	Endpoint	Código obtenido	Resultado
14	GET	/api/Usuario	200 OK	Correcto
15	GET	/api/Usuario/2	200 OK	Correcto
16	POST	/api/Usuario/Login	200 OK	Correcto

Evidencia detallada por endpoint

Listar categorías — GET /api/Categoría

Solicitud

GET http://apiwebm3c.somee.com/api/Categoria
Authorization: Bearer <token>

Respuesta — 200 OK

JSON


```
[
  { "id": 1, "nombre": "Aventura" },
  { "id": 2, "nombre": "Ciencia ficción" },
  { "id": 3, "nombre": "Misterio" },
  { "id": 4, "nombre": "Fantasía" },
  { "id": 5, "nombre": "Terror" },
  { "id": 6, "nombre": "Romance" },
  { "id": 7, "nombre": "Historia" },
  { "id": 8, "nombre": "Comedia" },
  { "id": 9, "nombre": "Acción" },
  { "id": 10, "nombre": "Drama" }
]
```


Listar historias — GET /api/Historia

Solicitud

GET http://apiwebm3c.somee.com/api/Historia
Authorization: Bearer <token>

Respuesta — 200 OK (fragmento del primer registro)

```JSON
[
  {
    "id": 1,
    "titulo": "La brújula de las mareas",
    "sinopsis": "Una tripulación busca una isla que aparece una vez cada cien
años.",
    "categorias": [
      { "id": 1, "nombre": "Aventura" },
      { "id": 2, "nombre": "Ciencia ficción" }
    ],
    "categoriaIds": null,
    "idCapituloInicial": 1,
    "capituloInicial": {
      "id": 1,
      "tipo": 0,
      "titulo": "El mapa roto",
      "texto": "El mapa señala dos rutas hacia la isla.",
      "opciones": [
        { "texto": "Seguir el arrecife", "idCapDestino": 2 },
        { "texto": "Confiar en la brújula", "idCapDestino": 3 }
      ]
    },
    "estado": 1
  }
]
```

Consultar catálogo por categoría — GET /api/Historia/Catálogo

Solicitud

GET http://apiwebm3c.somee.com/api/Historia/Catalogo?idCategoria=1
Authorization: Bearer <token>

Respuesta — 200 OK

```JSON
[
  {
    "id": 4,
    "titulo": "El bosque de cristal",
    "sinopsis": "Dos hermanos atraviesan un bosque que cambia según sus
decisiones.",
```

```
    "categorias": [
      { "id": 1, "nombre": "Aventura" },
      { "id": 4, "nombre": "Fantasía" }
    ]
  },
  {
    "id": 1,
    "titulo": "La brújula de las mareas",
    "sinopsis": "Una tripulación busca una isla que aparece una vez cada cien
años.",
    "categorias": [
      { "id": 1, "nombre": "Aventura" },
      { "id": 2, "nombre": "Ciencia ficción" }
    ]
  }
]
```

Ranking de historias — GET /api/Historia/Ranking

Solicitud

GET http://apiwebm3c.somee.com/api/Historia/Ranking
Authorization: Bearer <token>

Respuesta — 200 OK (fragmento)

JSON

```
[
  {
    "idHistoria": 1,
    "titulo": "La brújula de las mareas",
    "cantidadLecturasCompletadas": 5
  },
  {
    "idHistoria": 2,
    "titulo": "El último faro",
    "cantidadLecturasCompletadas": 4
  },
  {
    "idHistoria": 6,
    "titulo": "Cartas del tiempo",
    "cantidadLecturasCompletadas": 2
  }
```

```
]
```

Consultar finales de una historia — GET /api/Historia/{id}/Finales

Solicitud

GET http://apiwebm3c.somee.com/api/Historia/1/Finales
Authorization: Bearer <token>

Respuesta — 200 OK

```
JSON
{
  "idHistoria": 1,
  "tituloHistoria": "La brújula de las mareas",
  "finales": [
    {
      "idFinal": 3,
      "titulo": "Final: tesoro compartido",
      "cantidadVecesAlcanzado": 4,
      "esMasFrecuente": true
    },
    {
      "idFinal": 4,
      "titulo": "Final: tormenta eterna",
      "cantidadVecesAlcanzado": 1,
      "esMasFrecuente": false
    }
  ]
}
```

Consultar una historia por ID — GET /api/Historia/{id}

Solicitud

GET http://apiwebm3c.somee.com/api/Historia/1
Authorization: Bearer <token>

Respuesta — 200 OK (resumen)

```JSON
{
  "id": 1,
  "titulo": "La brújula de las mareas",
  "sinopsis": "Una tripulación busca una isla que aparece una vez cada cien años.",
  "categorias": [
    { "id": 1, "nombre": "Aventura" },
    { "id": 2, "nombre": "Ciencia ficción" }
  ],
  "idCapituloInicial": 1,
  "capituloInicial": {
    "id": 1,
    "tipo": 0,
    "titulo": "El mapa roto",
    "texto": "El mapa señala dos rutas hacia la isla.",
    "opciones": [
      { "texto": "Seguir el arrecife", "idCapDestino": 2 },
      { "texto": "Confiar en la brújula", "idCapDestino": 3 }
    ]
  },
  "estado": 1
}
```

Listar lecturas — GET /api/Lectura

Solicitud

GET http://apiwebm3c.somee.com/api/Lectura
Authorization: Bearer <token>

Respuesta — 200 OK (fragmento del primer registro)

```JSON
[
  {
    "id": 1,
    "idUsuario": 2,
    "usuario": {
      "id": 2,
      "nombre": "Bruno",
      "apellido": "Silva",
      "nombreUsuario": "lector01",
```

```
      "rol": 1,
      "email": "bruno.silva@obligatorio.test",
      "password": "HashDemo_lector01"
    },
    "idHistoria": 1,
    "historia": {
      "id": 1,
      "titulo": "La brújula de las mareas"
    },
    "fechaInicio": "2026-10-01T09:00:00",
    "fechaFin": "2026-10-01T09:35:00",
    "idFinal": 3,
    "finalAlcanzado": {
      "id": 3,
      "titulo": "Final: tesoro compartido"
    }
  }
]
```

Iniciar una lectura — POST /api/Lectura

Solicitud

POST http://apiwebm3c.somee.com/api/Lectura
Authorization: Bearer <token>
Content-Type: application/json

```
JSON
{
  "idHistoria": 1,
  "idUsuario": 10
}
```

Respuesta — 201 Created

Location: api/Lectura

```
JSON
{
  "idLectura": 27,
```

```
  "idHistoria": 1,
  "tituloHistoria": "La brújula de las mareas",
  "idCapituloInicial": 1
}
```

Consultar una lectura por ID — GET /api/Lectura/{id}

Solicitud

GET http://apiwebm3c.somee.com/api/Lectura/27?idUsuario=10
Authorization: Bearer <token>

Respuesta — 200 OK (resumen)

JSON


```
{
  "id": 27,
  "idUsuario": 10,
  "usuario": {
    "id": 10,
    "nombre": "Julian",
    "apellido": "Perez",
    "nombreUsuario": "lector09",
    "rol": 1,
    "email": "julian.perez@obligatorio.test",
    "password": "HashDemo_lector09"
  },
  "idHistoria": 1,
  "historia": {
    "id": 1,
    "titulo": "La brújula de las mareas"
  },
  "fechaInicio": "2026-11-09T15:17:38.6563995",
  "fechaFin": "2026-11-09T15:17:48.8989135",
  "idFinal": 3,
  "finalAlcanzado": {
    "id": 3,
    "titulo": "Final: tesoro compartido"
  }
}
```

Ranking de usuarios — GET /api/Lectura/RankingUsuarios

Solicitud

GET http://apiwebm3c.somee.com/api/Lectura/RankingUsuarios
Authorization: Bearer <token>

Respuesta — 200 OK (fragmento)

```JSON
[
  {
    "idUsuario": 2,
    "nombreUsuario": "lector01",
    "cantidadHistoriasCompletadas": 3
  },
  {
    "idUsuario": 5,
    "nombreUsuario": "lector04",
    "cantidadHistoriasCompletadas": 3
  },
  {
    "idUsuario": 10,
    "nombreUsuario": "lector09",
    "cantidadHistoriasCompletadas": 3
  },
  {
    "idUsuario": 3,
    "nombreUsuario": "lector02",
    "cantidadHistoriasCompletadas": 2
  }
]
```

Historial de lecturas por usuario y período — GET /api/Lectura/Historial

Solicitud

GET http://apiwebm3c.somee.com/api/Lectura/Historial?idUsuario=10&mes=11&anio=2026
Authorization: Bearer <token>

Respuesta — 200 OK

```JSON
[
  {
    "idLectura": 27,
    "historia": "La brújula de las mareas",
    "fechaInicio": "2026-11-09T15:17:38.6563995",
    "fechaFin": "2026-11-09T15:17:48.8989135",
    "completada": true,
    "finalAlcanzado": "Final: tesoro compartido"
  }
]
```

Consultar un capítulo de la lectura — GET /api/Lectura/{id}/Capitulo/{idCapitulo}

Solicitud

GET http://apiwebm3c.somee.com/api/Lectura/27/Capitulo/1?idUsuario=10
Authorization: Bearer <token>

Respuesta — 200 OK

```JSON
{
  "idLectura": 27,
  "idHistoria": 1,
  "tituloHistoria": "La brújula de las mareas",
  "capitulo": {
    "id": 1,
    "tipo": 0,
    "titulo": "El mapa roto",
    "texto": "El mapa señala dos rutas hacia la isla.",
    "opciones": [
      { "texto": "Seguir el arrecife", "idCapDestino": 2 },
      { "texto": "Confiar en la brújula", "idCapDestino": 3 }
    ]
  },
  "lecturaFinalizada": false
}
```

Avanzar en la lectura — POST /api/Lectura/{id}/Avanzar

Solicitud 1 — avanzar del capítulo 1 al capítulo 2

POST http://apiwebm3c.somee.com/api/Lectura/27/Avanzar
Authorization: Bearer <token>
Content-Type: application/json

```
JSON
{
  "idUsuario": 10,
  "idCapituloOrigen": 1,
  "idCapDestino": 2
}
```

Respuesta 1 — 200 OK

```
JSON
{
  "idLectura": 27,
  "idHistoria": 1,
  "tituloHistoria": "La brújula de las mareas",
  "capitulo": {
    "id": 2,
    "tipo": 0,
    "titulo": "La ruta del arrecife",
    "texto": "El barco atraviesa el arrecife y encuentra la isla.",
    "opciones": [
      { "texto": "Compartir el hallazgo", "idCapDestino": 3 },
      { "texto": "Buscar refugio", "idCapDestino": 4 }
    ]
  },
  "lecturaFinalizada": false
}
```

Solicitud 2 — avanzar del capítulo 2 al final 3

```
JSON
{
  "idUsuario": 10,
  "idCapituloOrigen": 2,
  "idCapDestino": 3
}
```

Respuesta 2 — 200 OK

```JSON
{
  "idLectura": 27,
  "idHistoria": 1,
  "tituloHistoria": "La brújula de las mareas",
  "capitulo": {
    "id": 3,
    "tipo": 1,
    "titulo": "Final: tesoro compartido",
    "texto": "La tripulación comparte el tesoro con su pueblo.",
    "opciones": null
  },
  "lecturaFinalizada": true
}
```

Listar usuarios — GET /api/Usuario

Solicitud

GET http://apiwebm3c.somee.com/api/Usuario
Authorization: Bearer <token>

Respuesta — 200 OK (fragmento de los primeros registros)

```JSON
[
  {
    "id": 1,
    "nombre": "Ana",
    "apellido": "Administrador",
    "nombreUsuario": "admin01",
    "rol": 0,
    "email": "ana.admin@obligatorio.test",
    "password": "HashDemo_Admin01"
  },
  {
    "id": 2,
    "nombre": "Bruno",
    "apellido": "Silva",
    "nombreUsuario": "lector01",
    "rol": 1,
    "email": "bruno.silva@obligatorio.test",
    "password": "HashDemo_lector01"
  }
]
```

Consultar un usuario por ID — GET /api/Usuario/{id}

Solicitud

GET http://apiwebm3c.somee.com/api/Usuario/2
Authorization: Bearer <token>

Respuesta — 200 OK

JSON


```
{
  "id": 2,
  "nombre": "Bruno",
  "apellido": "Silva",
  "nombreUsuario": "lector01",
  "rol": 1,
  "email": "bruno.silva@obligatorio.test",
  "password": "HashDemo_lector01"
}
```